ChatGPT

Developing a High-Accuracy Medical Chatbot: Data, Augmentation, Distillation, and Fine-Tuning

1. Public Datasets for Medical Dialogue and QA

To fine-tune our foundation model (MedGemma-27B) for a doctor-patient chatbot, we can leverage several **publicly available datasets** in the medical domain. These datasets provide question-answer pairs and dialogues covering symptoms, diagnoses, treatments, and prescriptions, which are vital for our use-case (accurate medical Q&A and advice). Key datasets include:

- **MedDialog (English)** – A large dataset of **~260,000 patient-doctor conversations** covering 96 medical specialties. These dialogues were scraped from online health consultation websites (HealthcareMagic and iCliniq) [1] [2]. Each dialogue typically starts with a patient's question or description of symptoms, followed by the doctor's answer or advice.
- **MedDialog (Chinese)** – An even larger Chinese medical dialogue dataset with **~1.1 million conversations** (172 specialties), useful if multilingual support is needed [2]. (MedDialog was originally introduced with 3.4M Chinese dialogues and 0.26M English dialogues [3] [4], with the English portion being the 260k noted above.)
- **COVID-Dialog** – A domain-specific subset of medical dialogues focused on COVID-19 and pneumonia. It includes **603 English consultations and ~1,393 Chinese consultations** about COVID-19 [5]. This dataset was released in 2020 during the pandemic to provide QA pairs on COVID-related issues [6]. While smaller, it's valuable if we want the chatbot to handle pandemic-related queries accurately.
- **HealthCareMagic and iCliniq** – These are the source forums for MedDialog's English data. In some releases they are available separately (e.g. **HealthCareMagic-100k** with 100,000 dialogues, and **iCliniq-10k** with 10,000 dialogues) [7]. However, using the combined MedDialog-EN is more convenient as it already aggregates these.
- **MedQuAD** – A **Medical Question Answering dataset of 47,457 QA pairs** obtained from 12 NIH (National Institutes of Health) websites [8]. These are high-quality *consumer health questions* and their answers, covering various conditions and treatments (e.g. FAQs from MedlinePlus, Cancer.gov, etc.). MedQuAD provides **reliable, fact-based Q&A pairs** that can help our model learn accurate medical information and how to answer patient queries with evidence-backed content.
- **MedMCQA** – A large-scale **multiple-choice question answering dataset with ~194,000 questions** from medical entrance exams (AIIMS and NEET-PG in India) [9]. Each question comes with 4 options and the correct answer (plus an explanation). This dataset spans 21 medical subjects and ~2,400 topics, testing everything from anatomy and physiology to pathology [9]. We can use MedMCQA by converting it into QA format – for example, taking the question and the correct answer (and possibly incorporating the explanation text as context or as part of the answer). Training on these **exam-style questions** can inject a broad range of medical knowledge and improve the model's accuracy on factual recall and reasoning (since many questions require reasoning to select the answer).
- **PubMedQA** – A dataset of **medical research question-answer pairs** derived from PubMed articles (focused on research findings). It contains 1,000 expert-labeled QA instances, plus ~61k unlabeled

and ~211k AI-generated QA instances [10]. Each question is based on a PubMed article, and the answer is typically "yes/no/maybe" or a short fact, along with an evidence snippet. While not in a patient-doctor format, PubMedQA can help the model learn to answer **factoid questions** with supporting evidence. It might improve the model's ability to handle questions about medical literature or factual queries.

- **DrugEHRQA** – A QA dataset focusing on **medication-related queries** in electronic health records. It contains **over 70,000 question-answer pairs** about medications, constructed from structured and unstructured EHR data (MIMIC-III) [11] [12]. For example, questions like *"What medication was the patient given for condition X?"* with answers extracted from patient records. This dataset can be useful for training the model on medication details – e.g., identifying drugs, dosages, and prescriptions from context – which is directly relevant to giving prescription advice. Although these are more database-oriented QAs, they can broaden the model's knowledge of drug names and typical uses.
- **MIRIAD** – *Medical Instruction and Retrieval Dataset*, a **massive newly-released dataset of medical Q&A pairs** generated in a semi-synthetic way (grounded on medical literature). MIRIAD contains *millions* of question-answer pairs (roughly 4.4–5.8 million after filtering) that were created by prompting GPT-3.5 on chunks of peer-reviewed medical articles [13]. Each Q&A is tied to a specific journal passage, ensuring the answer is grounded in a reliable source. This dataset is openly available and was shown to improve factual accuracy in medical QA [14] [13]. We can use MIRIAD to **fine-tune for factual correctness**, as it provides an unprecedented breadth of medical questions with evidence-backed answers. (It effectively gives our model access to knowledge from a huge swath of medical literature via supervised training.)
- **Other Niche Datasets** – There are various smaller datasets and benchmarks that could also be leveraged if needed:
- *HealthSearchQA* (approximately 3,000 common consumer medical questions, derived from search engine queries) – useful for seeing the types of questions laypeople ask [15].
- *Medical FAQ datasets* from specific sites or a compilation like **Consumer Health QA** – these often pair a user question with an answer from a medical professional or a trusted source (some overlap with MedQuAD sources).
- *Medical dialogue in specific domains* (e.g., **MediQA 2019** challenge had some consumer health questions and answers from NIH's online systems).
- *Doctor-patient interaction simulations* – for instance, a published OSCE simulation dataset of 272 transcribed physician-patient interviews focusing on respiratory ailments [16] [17]. Such data can be useful for *conversational nuance*, though in smaller quantity.
- *Medical exam QA benchmarks* like **MedQA USMLE** (questions from USMLE exams) or the medical category of **MMLU** – these can serve more for evaluation, but also provide tough questions for training. For example, MedQA (USMLE) is a set of USMLE exam questions which can test the model's diagnostic reasoning.

Why these datasets? Together, the above resources cover **both conversational dialogues and factual Q&A**. MedDialog provides the *conversational* format we need (including how a doctor explains diagnoses and treatments to a patient). MedQuAD, MedMCQA, MIRIAD, etc. provide a wealth of factual knowledge and question-answering ability on medical topics (ensuring the bot knows the correct information and can reason through medical questions). All these datasets are publicly available, which means we can use them freely for research/fine-tuning. By combining them, we will expose our model to a wide variety of medical queries – from casual patient questions to formal exam problems – and their answers, which should greatly enhance both its knowledge and its skill in answering with accuracy and proper reasoning.

2. Merging and Formatting Data for Fine-Tuning

Once we have gathered relevant datasets, the next step is to **merge and re-format the data into a consistent template** that our model can be fine-tuned on. Different datasets come in different formats (for example, dialogues vs. single-turn QA, or multiple-choice format). We need to convert them into a unified format, typically something like a **instruction → answer** pairing, possibly including context if available.

For our case, since we want a **QAC (Question-Answer-Context)** style, we can structure each training sample as a triple: - **Context:** any background information or dialogue context (if applicable), - **Question:** the user's question or prompt (e.g. patient's query), - **Answer:** the expected response (e.g. doctor's answer or explanation).

The "context" could be empty for single-turn QAs, or it could contain a brief patient background, case description, or previous dialogue turns if we want to fine-tune the model to handle multi-turn conversations. For example, if we have a multi-turn consultation from MedDialog, we might use the earlier dialogue as context and the latest patient question as the "question", with the doctor's reply as the "answer".

Unifying format example: We might choose a JSON structure where each data point looks like:

```
{
  "context": "<optional context or patient history>",
  "question": "<the user or patient question>",
  "answer": "<the doctor's answer>"
}
```

We will then serialize this to a JSONL (JSON Lines) file or similar, which the fine-tuning script can load. Alternatively, we could directly generate text prompt-response pairs (for example, constructing a single text string for each QA with special tokens or delimiters), but using a structured format (JSON/CSV) is convenient for programmatic assembly and for feeding into libraries like HuggingFace's `datasets`.

Below is a **script snippet** (in Python) demonstrating how we can merge multiple dataset sources and output a unified training file. This pseudocode assumes we have loaded or have access to each dataset in a Python-friendly format (such as lists of dictionaries). We will: 1. Iterate through each dataset, 2. Convert each entry to the unified format, 3. Optionally filter or clean the data (e.g., remove unanswered questions or overly long responses), 4. Append to a master list, 5. Save the combined data to disk.

```
# Pseudocode for merging datasets into a unified QAC format
import json

merged_data = []

# Example 1: Process MedDialog (English) dataset
# Assume meddialog_data is a list of dialogues, each as a list of turns
# (where each turn has 'speaker' and 'text', or similar structure)
```

```
for dialog in meddialog_data:
    # For simplicity, take the first patient question and first doctor answer
    # (We could also take last Q&A, or even create multiple Q-A pairs from a
long conversation)
    for i in range(len(dialog) - 1):
        if dialog[i]['speaker'] == 'patient' and dialog[i+1]['speaker'] ==
'doctor':
            question = dialog[i]['text']
            answer = dialog[i+1]['text']
            merged_data.append({
                "context": "",  # no additional context for single-turn Q&A
                "question": question.strip(),
                "answer": answer.strip()
            })
    # (We could break after one Q-A per dialogue to avoid overweighting,
    #  or use multiple QA from same dialogue as separate entries.)

# Example 2: Process MedQuAD dataset (already QA pairs, possibly with some
context info)
for entry in medquad_data:
    question = entry['question']
    answer = entry['answer']
    # Some MedQuAD entries might have an associated source or context snippet
    context = entry.get('context', "")  # use provided context if available
    merged_data.append({
        "context": context.strip(),
        "question": question.strip(),
        "answer": answer.strip()
    })

# Example 3: Process MedMCQA dataset (multiple-choice questions)
for item in medmcqa_data:
    question = item['question']
    correct_option = item['correct_answer']        # e.g., "A" or the text of
the correct answer
    explanation = item.get('explanation', "")      # if available
    # We'll form the answer as the correct option plus explanation (to make a
full answer).
    answer_text = item['options'][correct_option]
# get the text of the correct option
    if explanation:
        answer_text = answer_text + " - Explanation: " + explanation
    merged_data.append({
        "context": "",  # these are standalone Qs, no extra context
        "question": question.strip(),
        "answer": answer_text.strip()
    })
```

```
# Example 4: Process a dialogues dataset with context, e.g., multi-turn from
MedDialog or other
for dialog in long_dialogs_data:

# Use the entire dialogue history except last turn as context, and last turn Q-
>A as pair
    # (assuming last turn is doctor answer and second last is patient question,
adjust as needed)
    if len(dialog) >= 2:
        context_turns = dialog[:-1]  # all except last turn
        question_turn = dialog[-1]   # last turn (maybe patient's question or
doctor? adjust logic)
        # If last turn is doctor, then the question is second last.
        # Here, assume dialogues alternate and last turn is doctor's answer.
        if question_turn['speaker'] == 'doctor':
            # find the last patient question before it
            for j in range(len(dialog)-1, -1, -1):
                if dialog[j]['speaker'] == 'patient':
                    question_text = dialog[j]['text']
                    break
            answer_text = question_turn['text']
        else:
            # If last turn is patient, then model should answer that
            question_text = question_turn['text']
            answer_text = ""  # (no provided answer in data, skip or generate
via teacher)
            continue  # skip if we don't have an answer
        # Combine context (all previous turns) into a single string
        ctx_str = ""
        for turn in context_turns:
            ctx_str += f"{turn['speaker'].capitalize()}: {turn['text']}\n"
        merged_data.append({
            "context": ctx_str.strip(),
            "question": question_text.strip(),
            "answer": answer_text.strip()
        })

# (Additional processing for other datasets like PubMedQA, DrugEHRQA, etc.,
similarly.)
# After processing all sources:
print(f"Total merged samples: {len(merged_data)}")

# Save to a JSONL file
with open("merged_medqa_data.jsonl", "w") as f:
    for entry in merged_data:
        json_line = json.dumps(entry, ensure_ascii=False)
        f.write(json_line + "\n")
```

In the above script: - We handled **MedDialog** by pairing patient questions with doctor answers. Because MedDialog dialogues can be multi-turn, one strategy is to extract the first question and first answer of each dialogue, treating them as an independent QA pair. (Alternatively, we could use every QA exchange as separate entries, but be careful not to overweight long conversations too much). - **MedQuAD** was straightforward (already question and answer text). - **MedMCQA** required converting multiple-choice to direct QA. We took the correct option's text as the answer, and optionally appended the official explanation to enrich the answer. This way the model learns not just the letter of the answer but also the reasoning or additional info. - We showed how one might incorporate **context** for multi-turn dialogues: concatenating previous dialogue turns as a context string. The model then sees a prompt that includes the prior conversation, followed by the latest question it needs to answer. This would train it to handle conversations in context (remembering earlier details). If our use-case expects multi-turn dialogues, including context like this is very important.

Template formatting: When fine-tuning, we often wrap the input and output into a prompt template that the model was trained on or expected to see. For example, if MedGemma-27B is based on LLaMA or another model that uses `<s>User: ...</s><s>Assistant: ...</s>` style tokens, we should format accordingly. A simple approach for our dataset might be to present it in an *instruction-following format*. For instance:

- **Without explicit roles:** "**Question:** {question}\n**Context:** {context}\n**Answer:** {answer}". (If context is empty, we omit that line.)
- **With roles:** We can use a conversation format like:

  ```
  <|user|> Patient: {question plus any context if needed}<|end|>
  <|assistant|> Doctor: {answer}<|end|>
  ```

  This would mimic a conversation between patient and doctor. Some models (like LLaMA-based) use special tokens like `<s>` or `<|im_start|>` for roles, whereas others simply learn from textual cues (like lines starting with "Patient:" and "Doctor:").

Since our fine-tuning will be custom, we can choose a template that is intuitive. The key is to use the *same template consistently* for all training examples, so the model learns the expected format. For example, a unified prompt format might be:

```
[CONTEXT]
Patient: [question]
Doctor: [answer]
```

Where `[CONTEXT]` is replaced with something like "Patient history: ..." or previous turns if available (or removed if not applicable). We should document this format because we will use the same format when interacting with the model after fine-tuning.

Finally, after merging, we'll have one large file (or dataset object) containing all the training Q-A pairs. This "centralized" dataset is what we'll feed into the fine-tuning process. We should shuffle it to mix questions from different sources (ensuring the model doesn't learn to specialize only in one style before seeing

others). We may also want to **hold out a portion as a validation set** (for monitoring training loss and for benchmarking as discussed later).

Note: It's often useful to up-sample or weight certain parts of the data. For example, we might have millions of QA from MIRIAD (very fact-heavy) and only ~260k from MedDialog (conversational). To make sure the model learns to be conversational and not just a fact-reciting machine, we might give slightly higher sampling weight to dialogue data during training. Balancing the dataset can be important (as Unsloth suggests, e.g., using 75% reasoning data and 25% direct answer data to maintain reasoning skills [18]). We can achieve this by duplicating some entries or using a sampling probability in the data loader.

3. Data Augmentation Techniques for Questions

To improve our model's robustness and cover more variations of user inputs, we should **augment our dataset**. Data augmentation in NLP often involves generating paraphrases or semantically equivalent variations of the existing queries. In our scenario, the questions asked by users/patients could be phrased in many ways. If we augment the questions, the model can learn to handle those variations without getting confused. Here are some strategies:

- **Paraphrasing Questions via an LLM:** We can use a strong language model (possibly the teacher model or another high-quality model) to rephrase each question in our dataset. For instance, if the original question is *"I have a headache and a fever, what should I do?"*, a paraphrase could be *"I'm experiencing fever and head pain; what treatment do you recommend?"* – different words, same intent. We could prompt an LLM with something like: *"Paraphrase the following medical question in a different way while preserving its meaning: '<question>'"*. Another prompt could be: *"Generate 2 alternative ways a patient could ask: '<question>'"*. By doing this, for each original Q, we might get 2-3 augmented questions. We would then pair each augmented question with the **same answer** as the original, assuming the meaning hasn't changed. This effectively multiplies our training examples. Using an LLM for paraphrasing is powerful because it can capture subtle rephrasings and maintain medical terminology. (In 2024–2025, advanced paraphrase models or LLMs became widely used for data augmentation [19].)
- **Splitting complex questions:** Often, patients (or exam questions) might bundle multiple sub-questions or a long complex query. For training, it can help to *split these into smaller chunks*. For example, the multi-part question *"What could be causing my headache and stomach ache, and what medication should I take for them?"* could be split into two questions: (1) "What are possible causes for having a headache and stomach ache together?" and (2) "What medication can help with headaches and stomach aches?" – each with an appropriate part of the answer. We might need to also split the answer accordingly or repeat the full answer for both. This augmentation teaches the model to answer focused questions on a single aspect as well. We can do this splitting manually for some, or even prompt an LLM: *"Split the following question into two or more questions that each focus on a single issue, yet collectively cover the original query."*
- **Context perturbation:** If a question comes with context (say a patient history), we can create variations of the context. For instance, if context says "45-year-old male with diabetes," we could create an alternate context like "50-year-old male with a history of diabetes" or "45-year-old female with diabetes" to ensure the model doesn't latch on to one specific detail too rigidly. We must be careful to keep the medical logic consistent (e.g., if the context gender is changed, the answer might differ for gender-specific conditions).

- **Back-translation:** This is a classic NLP augmentation: translate the question to another language and back to English. The result is usually a paraphrase. For example, English → French → English might turn "What are the symptoms of a kidney stone?" into "What symptoms does a kidney stone cause?" This can be done with machine translation APIs or models. This method can produce varied phrasing, though one must check that medical nuances aren't lost in translation.
- **Synonym replacement & phrasing tweaks:** On a smaller scale, one could automatically replace some terms with synonyms (e.g., "medication" -> "medicine", "pain" -> "ache") or change sentence structures (active/passive voice). However, medical context needs caution; not all synonyms are truly equivalent (e.g., "cold" vs "flu" are different conditions). So a controlled approach is needed. One could use a library like NLPAug or TextAttack to do random swaps, insertions, etc., but manual or LLM-guided paraphrasing is often higher quality for critical domains like this.

Using an **LLM to augment data** is a modern, effective approach. We do need to ensure the augmented questions **preserve the original intent**. It's wise to review some augmented examples or even have rules to catch distortions. For example, if the LLM paraphraser accidentally changes "no allergies" to "some allergies" in context, that would change the answer. One technique is to have the LLM only *rephrase* and explicitly instruct it not to add or remove facts.

We can incorporate augmentation into our preprocessing script. For example:

```
import openai  # as an example if using OpenAI API for augmentation

openai.api_key = "YOUR_API_KEY"

def paraphrase_question(question, n=2):
    prompt = f"Paraphrase the following medical question in {n} different ways,
while keeping the meaning the same:\n\"{question}\""
    response = openai.ChatCompletion.create(
        model="gpt-4",  # or use Gemini/NVIDIA teacher model via their API
        messages=[{"role": "user", "content": prompt}]
    )
    # Parse the response to extract the N paraphrases (assuming the model lists
them)
    paraphrases = []
    if response:
        text = response['choices'][0]['message']['content']
        # If the model enumerates paraphrases as a list, split them
        for line in text.split("\n"):
            line = line.strip("- ")  # remove bullet if any
            if len(line) > 5:
                paraphrases.append(line)
    return paraphrases[:n]

# Augment the questions in merged_data
augmented_data = []
for entry in merged_data:
    q = entry['question']
```

```
    a = entry['answer']
    ctx = entry['context']
    # Generate 2 paraphrased versions of the question
    try:
        new_questions = paraphrase_question(q, n=2)
    except Exception as e:
        new_questions = []
    for q2 in new_questions:
        augmented_data.append({
            "context": ctx,
            "question": q2,
            "answer": a
        })
# Combine with original data
merged_data_extended = merged_data + augmented_data
```

In this snippet, we used an OpenAI GPT-4 API call for demonstration. In practice, since we have teacher models (Gemini, etc.) accessible, we could use those via their API in a round-robin (to avoid rate limits as discussed later) to do the paraphrasing tasks. We should use whichever model provides the best quality paraphrases. A smaller model from NVIDIA might be sufficient for simple rephrasing, but to ensure medical nuance is preserved, using a top-tier model (like GPT-4 or Gemini if it's comparable) is safer.

Another augmentation to consider is generating entirely new Q&A pairs using an LLM – essentially **data synthesis**. This borders on knowledge distillation (next section) when done with a teacher model. For example, we could prompt the teacher to *invent* a realistic patient question and then answer it, yielding new training data beyond our original datasets. This is how Stanford's Alpaca dataset was built – they generated 52k instruction-output examples using OpenAI's model to fine-tune a smaller model [20]. For our case, we could instruct the teacher to generate dialogues or QA in certain areas we feel underrepresented (e.g., rare diseases, specific prescription questions). This method should be used carefully – we'd want to ensure the teacher's generated answers are factually correct (since the student will learn from them blindly). However, with a high-quality teacher and possibly some human review, this can significantly enrich the training set.

In summary, data augmentation will **enrich the diversity of phrasing** the model sees: - The model won't be thrown off by a question phrased in an unusual way because it likely saw a paraphrase of it during training. - It might generalize better to new questions and have improved recall of edge cases due to exposure to more variants.

4. Knowledge Distillation from Teacher Models

Besides augmenting existing data, we can leverage **knowledge distillation** to improve our student model (MedGemma-27B) by having it learn directly from one or multiple **teacher models**. In this project, we have access to large, powerful models via APIs – specifically, the **Gemini API** (with models like `"gemini-2.5-flash"` and `"gemini-2.5-flash-lite-preview"`) and an **NVIDIA API** (with various smaller and larger LLMs). These will serve as our "teachers," meaning we will use them to generate high-quality answers or reasoning, and then fine-tune our student on those outputs.

There are a few forms this distillation can take:

- **Response Distillation (Direct Q→A):** For each question in our dataset (or newly generated ones), we can ask a teacher model to produce an answer (and perhaps an explanation). We then use the teacher's answer as the target output for the student to learn. The student essentially **imitates** the teacher's answers. This is analogous to how Alpaca was trained by imitating OpenAI's text-davinci outputs [20]. Because the teacher is presumably very accurate and comprehensive, its answers can elevate the quality of the student's responses. For example, if a dataset answer is short or slightly outdated, the teacher (which might be more up-to-date or detailed) could generate a richer answer. We would then prefer the student learns that richer answer. Over a large number of QA pairs, the student model's knowledge and style will shift closer to the teacher's.
- **Chain-of-Thought Distillation:** We can have the teacher provide not just a final answer but a **step-by-step reasoning (rationale)** for each question. This is often done by prompt engineering (e.g., telling the teacher *"Think step by step"* or using few-shot examples to produce a chain-of-thought). The result might look like: *"Question…Solution: First, this symptom indicates X… therefore, the diagnosis is Y. Answer: The patient likely has Y and should do Z."* Training the student on these detailed explanations can teach it to **reason through problems and produce well-justified answers**. Research has shown that giving a smaller model the rationales from a larger model can significantly improve the smaller model's reasoning performance [21] [22]. The rationales act as an additional training signal, helping the student learn intermediate reasoning patterns that it might not infer from final answers alone. In practice, we can format the training data to include the chain-of-thought: e.g., treat the entire rationale+answer as the target text the student must generate given the question. After such fine-tuning, the student might start producing its own chain-of-thought when asked, or at least go through a better internal reasoning process. *(As a note, one can either keep the chain-of-thought hidden or prompt the model to output it – we might choose to have an* internal *chain-of-thought for reasoning but only show the final answer to users, depending on design.)*
- **Knowledge Transfer via Unlabeled Data:** If we have a lot of unlabeled user questions (or we anticipate certain types of questions but don't have answers), we can use the teacher to label them. For example, we could compile a list of plausible questions (like "What's a good diet for diabetes?") and have the teacher answer them. This extends our training set beyond the original datasets. The key is the questions should be relevant and diverse. We could generate these questions either by brainstorming, using real user logs (if available), or even asking the teacher to come up with questions ("self-questioning"). This is another way to do **self-instruct** or synthetic data generation.

Implementing knowledge distillation: 1. **Select the teacher(s) and prompts:** We will use the **Gemini model** (likely a powerful general LLM, possibly analogous to GPT-4 in capability if it's called "Gemini") and possibly a large NVIDIA-served model (NVIDIA has released models like Megatron-Turing NLG, etc., or offered via NeMo framework; we'd pick one known for good quality). We should verify which model from NVIDIA's API would be best as a teacher – maybe a domain-tuned one if available (e.g., a BioMedGPT or a large general model). Since the user specifically suggests using "Gemini" and an NVIDIA model, we assume these are reliable. 2. **Generate answers with the teacher:** For each question in our **training set (and any new questions we add)**, we call the teacher API to get an answer. We might engineer the prompt to ensure the teacher's answer is in the format we want the student to learn (for instance, include a request for a concise answer with sources if we want sourced answers). Because our chatbot should be *accurate and well-sourced*, we could even prompt the teacher to cite a source or rationale in its answer. If the teacher model supports retrieval (like maybe Gemini can search, not sure), we could incorporate that. At minimum, we can prompt: *"You are a doctor. Answer the question with accurate information and reasoning. If relevant, mention the source of your information."* This might cause the teacher to include a reference (or at least a plausible

reference). The student would then learn to include sources (though verifying them is another matter). 3. **Prepare the training pairs:** We will have the same question, but now the **target answer is the teacher's answer** (which may be superior to the original dataset answer). Essentially we replace or augment the ground-truth answers with the teacher's outputs. It's often a good idea to **mix** original answers with teacher answers, instead of completely replacing. The original answers might be shorter or less detailed but also they might be more directly to the point; including both gives the student a mix. However, if the teacher is very reliable, we might prioritize its answers. 4. **Iterative refinement:** We can iterate this process. For example, fine-tune the student on the first wave of distillation, then possibly query the teacher again for any questions the student still gets wrong or for new questions to push it further. This is not strictly necessary, but sometimes a second distillation round (or mixing multiple teachers) can help. Since we have *multiple teacher APIs*, we could even do an **ensemble distillation**: get answers from Gemini and from the NVIDIA model and perhaps even from GPT-4 if available, then use all of those as training examples. That could teach the student a style that is a blend or the best of each.

A concrete example of distillation in action is **Stanford Alpaca**, where OpenAI's text-davinci-003 was used to generate 52k instruction-response pairs, and the LLaMA model fine-tuned on that matched a lot of GPT-3.5's capabilities [20] . Similarly, we want MedGemma to absorb the knowledge of Gemini (and others). Distillation is effectively **making the student mimic the teacher**.

One more advanced trick: using **preference-based distillation**. Instead of just giving one teacher answer, we could generate multiple answers and rank them (or use a preference model). But since the user hasn't asked specifically, we can keep to simpler supervised distillation.

Knowledge distillation using our APIs – pseudo-code:

```
# Pseudocode to use teacher models (Gemini, NVIDIA) to create training data
gemini_keys = [os.getenv(f"Gemini_COS30018_{i}") for i in range(1,6)]
nvidia_keys = [os.getenv(f"NVIDIA_COS30018_{i}") for i in range(1,6)]
gemini_index = 0
nvidia_index = 0

def call_gemini(question):
    global gemini_index
    api_key = gemini_keys[gemini_index]
    gemini_index = (gemini_index + 1) % len(gemini_keys)
    prompt = f"Patient asks: {question}\nDoctor answers (with reasoning and
accurate info):"
    # ... call Gemini API with the prompt and api_key ...
    response = call_gemini_api(prompt, api_key=api_key, model="gemini-2.5-
flash")
    return response

def call_nvidia(question):
    global nvidia_index
    api_key = nvidia_keys[nvidia_index]
    nvidia_index = (nvidia_index + 1) % len(nvidia_keys)
```

```
    prompt = f"Q: {question}\nA:"  # simple prompt or similar instruction
    response = call_nvidia_api(prompt, api_key=api_key, model="best-model")
    return response

distilled_data = []
for entry in merged_data:
    q = entry['question']
    # Use teacher models to get answers
    try:
        ans1 = call_gemini(q)
    except Exception:
        ans1 = None
    try:
        ans2 = call_nvidia(q)
    except Exception:
        ans2 = None
    # If we got responses, choose one or both
    if ans1:
        distilled_data.append({"context": entry['context'], "question": q,
"answer": ans1})
    if ans2:
        distilled_data.append({"context": entry['context'], "question": q,
"answer": ans2})
```

In practice, we might just use one teacher at a time to maintain consistency in style. But nothing stops us from using multiple and mixing. If doing so, we should be aware that different models have different styles (one might be more verbose, one might use more technical language). A mixture could make the student a bit unpredictable in style. To mitigate that, we can edit or normalize the teacher outputs (e.g., always instruct them to follow a certain style in the prompt).

Also, **quality control** is crucial. Large models are very good, but they can still produce the occasional hallucination or error. We don't want the student to learn incorrect facts. So we might want to review a subset of the distilled answers for obvious mistakes. If the teacher cites sources, verify a couple. If any issues, either fix those answers or drop them from training. The advantage is that we are the ones generating this data, so we have some control.

Finally, after distillation, our student model should hopefully approach the teacher's performance but at a fraction of the size. This addresses the goal of *optimal accuracy*: by leaning on a very accurate teacher, we transfer as much of that accuracy as possible to the student. Empirically, distilled models often achieve accuracy close to the teacher on the domains they were distilled on [23] [21], though usually not exceeding the teacher.

One more note: we might perform *distillation with chain-of-thought* as mentioned. Google researchers describe a "Distilling step-by-step" method where they extracted rationales from a big model and used them to supervise a smaller model, yielding significant performance gains on reasoning tasks [21] [22]. We can apply that here by capturing the teacher's reasoning. For example, prompt Gemini: *"Answer with step-by-step reasoning:"* and include that in the answer text. Our fine-tuning data would then have answers that contain

the reasoning process. Over time, MedGemma-27B will learn to produce similar reasoning. This can greatly improve its **reasoning quality**, which is one of our key goals.

5. Fine-Tuning with LoRA and GRPO for Improved Reasoning

With our training data ready (original + augmented + distilled), we move on to the **fine-tuning step**. Fine-tuning a 27B model is computationally heavy, so we will likely use **parameter-efficient fine-tuning techniques** like **LoRA (Low-Rank Adaptation)**, and possibly advanced training algorithms like **GRPO** to further enhance reasoning and alignment.

LoRA (Low-Rank Adaptation): LoRA is a method that inserts trainable low-rank matrices into the model's layers and **only trains those**, instead of updating all 27B parameters [24] [25]. This drastically reduces VRAM and time requirements. It's well-suited for fine-tuning large models on custom tasks because it's efficient and you can later remove or swap out the LoRA modules to get back the original model if needed. Many LLM fine-tuning projects (Alpaca, etc.) use LoRA or QLoRA. In our case, using LoRA means: - We load the MedGemma-27B base model (likely in 16-bit or 8-bit precision to save memory). - We **attach LoRA adapters** to key weight matrices (typically the query, key, value, and output projection of each transformer layer, and maybe others like the feed-forward projections) [26]. - We initialize those LoRA weights (usually small matrices) and set up training so that only those are trainable (the rest of the model stays frozen). - During fine-tuning, the gradients only update the LoRA parameters, which encode the difference needed for the new task. - After training, we can merge the LoRA weights into the base model or keep them separate. Keeping them separate allows toggling between the base and fine-tuned behavior easily.

Using LoRA can reduce memory by 4x or more and still achieve nearly full fine-tuning quality [24]. For a 27B model, full fine-tuning might be impractical on a single GPU, but LoRA fine-tuning can be done on a 24GB or 48GB GPU (or smaller if using 8-bit quantization of base model).

Setting up LoRA (example with HuggingFace PEFT):

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments,
Trainer

# Load base model in 8-bit to save memory (requires bitsandbytes library)
model = AutoModelForCausalLM.from_pretrained("MedGemma-27B", device_map="auto",
load_in_8bit=True)
tokenizer = AutoTokenizer.from_pretrained("MedGemma-27B")

# Prepare LoRA configuration – adjust the target modules based on model
architecture
lora_config = LoraConfig(
    r=16,  # rank of the LoRA matrices
    lora_alpha=32,  # scaling factor
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  # for example,
adapt all attention projection matrices
    bias="none",
```

```
    task_type="CAUSAL_LM"  # we're fine-tuning a causal language model
)
model = get_peft_model(model, lora_config)
print("LoRA parameters:", sum(p.numel() for p in model.parameters() if
p.requires_grad))
# At this point, only the LoRA params are trainable, which should be a small
fraction of 27B.

# Prepare our dataset for training (assuming we have it as `train_dataset`)
# For example, using HuggingFace Datasets:
# train_dataset = Dataset.from_json("merged_medqa_data.jsonl")
# But we might need to tokenize it:
def tokenize_example(example):
    prompt = ""
    if example["context"]:
        prompt += f"Context: {example['context']}\n"
    prompt += f"Question: {example['question']}\nAnswer:"
    # The model should generate the answer after 'Answer:'.
    # We combine prompt+answer as the training text, shifting the answer as the
label part.
    full_text = prompt + " " + example["answer"]
    tokens = tokenizer(full_text, truncation=True, max_length=1024)  #
max_length depends on context size
    # Could also prepare labels to ignore context part etc. For simplicity,
treat all as one sequence.
    return tokens

tokenized_train = train_dataset.map(tokenize_example,
remove_columns=train_dataset.column_names)

# Define training arguments
training_args = TrainingArguments(
    output_dir="medgemma_finetune",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    num_train_epochs=3,
    learning_rate=1e-4,
    fp16=True,
    logging_steps=50,
    save_steps=500,
    report_to="none"
)

# Initialize Trainer and train
trainer = Trainer(model=model, args=training_args,
train_dataset=tokenized_train)
trainer.train()
trainer.save_model("medgemma-27b-med-finetuned")
```

The above is a high-level example. We'd adjust details like batch size and learning rate based on experiments or validation loss. Also, if the dataset is huge (millions of examples with MIRIAD), we might not do many epochs – maybe even 1 epoch or a fraction is enough. We should monitor the model so it doesn't overfit (especially on smaller sets like MedDialog).

GRPO (General Reinforcement Pretraining Optimization): After (or during) supervised fine-tuning, we can apply GRPO to further refine the model's reasoning and alignment. GRPO is a reinforcement learning approach that combines ideas from PPO (Proximal Policy Optimization) and DPO (Direct Preference Optimization) [27]. In simpler terms, GRPO allows us to define a **reward function** for the model's outputs and then update the model so as to maximize that reward, using RL techniques. The Unsloth notebooks specifically mention using GRPO to train Qwen (and presumably Gemma models) for reasoning tasks with a *proximity-based reward* [28]. "Proximity-based" likely means the reward is higher if the model's answer is closer to a reference answer.

How this works for us: - We have a set of evaluation questions (or we can reuse training questions) with correct answers (ground truth). We define a reward function that compares the model's answer to the correct answer. For example, a simple reward could be **1 if the answer is exactly correct, 0 otherwise** [29]. Or more softly, the reward could be a similarity score between the answer and the reference (using BLEU or BERTScore as the measure). Unsloth's proximity reward might involve regex matching for exact math answers [30] or embedding similarity for open text. The idea is to give a higher score the "closer" the model output is to the desired answer. - We then have the model generate answers to the questions (its current policy). We compute the reward for each output. Using GRPO (which is like PPO but simplified for this use-case), we adjust the model weights to increase the probability of high-reward outputs and decrease probability of low-reward outputs. - GRPO training happens in *episodes* but since we have a fixed set of Q&A, it's more like supervised fine-tuning with a special loss. Essentially, it's doing something similar to policy gradient: if the model output gets a good score, tweak weights to make those kinds of outputs more likely in the future.

One can use HuggingFace's TRL (Transformer Reinforcement Learning) library which supports PPO and related algorithms. In the earlier snippet from the Medium article, they used `trl.GRPOTrainer` [31] [32]. We would plug in our model and a reward function.

For example, a **reward function** in code could be:

```
# Example reward: exact match
def reward_func(output: str, reference: str) -> float:
    return 1.0 if output.strip().lower() == reference.strip().lower() else 0.0
```

Or a more nuanced one:

```
import difflib
def reward_func(output: str, reference: str) -> float:
    seq = difflib.SequenceMatcher(None, output.lower(), reference.lower())
    return seq.ratio()  # returns a similarity between 0 and 1
```

Or even use an embedding:

```
import torch
from sentence_transformers import SentenceTransformer
bert_model = SentenceTransformer('all-MiniLM-L6-v2')
def reward_func(output: str, reference: str) -> float:
    emb_out = bert_model.encode(output, convert_to_tensor=True)
    emb_ref = bert_model.encode(reference, convert_to_tensor=True)
    cos_sim = torch.nn.functional.cosine_similarity(emb_out, emb_ref, dim=0)
    return float(cos_sim.cpu().numpy())
```

This would give a higher reward if the output is semantically closer to the reference answer.

With the reward function ready, we would initialize GRPO training. Unsloth's documentation suggests doing a **pre-fine-tuning** pass to ensure the model outputs are formatted correctly before applying GRPO (since RL can sometimes derail output formatting) [30]. We have essentially done that by supervised fine-tuning. So the model is already reasonably good.

Now, using TRL:

```
from trl import GRPOTrainer, GRPOConfig

# Prepare dataset for RL (list of prompts and reference answers)
prompts = []
references = []
for ex in rl_dataset:  # rl_dataset could be a subset of our data or separate
eval set
    user_prompt = ""
    if ex["context"]:
        user_prompt += f"{ex['context']}\n"
    user_prompt += ex["question"]
    prompts.append(user_prompt)
    references.append(ex["answer"])

# Define a reward function for the trainer (using our earlier reward_func)
def compute_reward(outputs, references):
    # outputs and references are lists of strings
    rewards = []
    for out, ref in zip(outputs, references):
        rewards.append(reward_func(out, ref))
    return rewards

# Configure GRPO
grpo_config = GRPOConfig(
    learning_rate=5e-6,
    # other parameters like batch_size, etc.
```

```
)
trainer = GRPOTrainer(model=model, tokenizer=tokenizer,
                      dataset=list(zip(prompts, references)),
                      reward_function=compute_reward,
                      grpo_config=grpo_config)
# Note: This is conceptual; actual TRL usage may differ in API.
trainer.train()
```

During GRPO training, the model will generate an answer for each prompt, get a reward, and update itself. Over many iterations, it learns to output answers that score well – i.e., that match the correct answers closely. This process, in effect, **fine-tunes the model's outputs to be more accurate**. It's like telling the model "You thought the answer might be X, but the correct answer was Y, adjust yourself to say Y with higher probability next time."

GRPO is particularly useful for **alignment and formatting** too. For instance, we can include penalties in the reward for certain undesired outputs (like if the model says something not factual or unsafe, reward = -1). In our context, we could incorporate a component in the reward for citing sources (if we want that). E.g., reward += 0.2 if the output contains a reference string "[" or a citation pattern. We have to be careful not to over-optimize weird metrics, but it's doable.

Unsloth's advanced notebook for Qwen3-4B suggests GRPO helped make it a *"reasoning model"*, with improved mathematical reasoning by rewarding correct step-by-step solutions [28]. We can adapt that idea for medical reasoning: for example, reward the model if it correctly reasons through a differential diagnosis. Defining that reward is hard though – perhaps we could have an automated checker or use the teacher model as an evaluator (e.g., ask teacher if student's answer is correct, then reward accordingly). That becomes a form of **reinforcement learning from AI feedback**. It's an area we could explore if time permits (having an LLM judge the student's answer quality).

To summarize, our fine-tuning pipeline will likely be: 1. **Supervised fine-tuning (SFT)** with LoRA on the merged/augmented/distilled dataset – to instill all the knowledge and base skills. 2. **GRPO (RL fine-tuning)** on a targeted set of tasks or questions – to polish the model's performance, especially on reasoning-intensive problems and to ensure answers align with ground truth. This stage can improve factual correctness by directly optimizing it [33]. It also can help maintain the **reasoning chains**, as we can reward the presence of a good reasoning chain if desired.

Both LoRA and GRPO are compatible: we can do GRPO on top of the LoRA fine-tuned model. In practice, we might incorporate LoRA in the RL trainer as well (so it only updates LoRA params). The medium guide shows using LoRA with GRPO by wrapping the model with `FastLanguageModel` and `get_peft_model` then using `GRPOTrainer` [34] [31].

One important detail: After fine-tuning, we'll have our **MedGemma-27B** model fine-tuned (with LoRA adapters). We should test it on some sample questions (especially ones it struggled with before or some from our validation set) to verify qualitatively that it's giving good, detailed answers with reasoning and correct info. It's possible that the model might become too verbose or too "teacher-like" after distillation. If needed, we can calibrate its verbosity via prompts or small additional training (e.g., include some instructions where the answer should be brief to teach it brevity when needed).

6. Benchmarking the Fine-Tuned Model

After fine-tuning, we need to **benchmark our model's performance** to ensure it meets the accuracy and reasoning quality we aim for. Benchmarking a medical QA chatbot can be done in several ways, combining **quantitative evaluation** on datasets with **qualitative assessment** of reasoning. We want to use *real datasets* for evaluation so that our claims about the model's performance are grounded in actual performance on known benchmarks (as the user mentioned, factual correctness and reasoning quality "have to be based on a real dataset" for benchmarking).

Here are some approaches to benchmarking, and we can use them in tandem:

- **Standard QA Metrics on Held-out Data:** We can take one or more of the datasets we collected and set aside a test portion that the model hasn't seen during training. For example, use 10% of MedDialog and MedQuAD as a test set, or use the official test sets if available (MedMCQA has an official test set split; PubMedQA has a standard dev/test; etc.). Then, we have the model answer each question, and we compare its answers to the reference answers. For evaluation:
- If the questions are *multiple-choice* (like MedMCQA or MedQA), we can simply check if the model's chosen answer (or the content of its answer) matches the correct option. Here we measure **accuracy** (% of questions answered correctly).
- If the questions are *free-response* (like MedDialog or MedQuAD where answers are explanatory text), we need a similarity metric. Common metrics include **Exact Match** (strict – rarely useful unless answers are one-word factual answers), **BLEU or ROUGE** (which measure n-gram overlap), and **BERTScore** or other embedding-based metrics (which measure semantic similarity) [35]. In the medical QA context, a loose overlap metric like ROUGE-L or an F1 score on key terms can be used. For example, the MMLU benchmark (Massive Multitask Language Understanding) when evaluating open-ended answers often uses an accuracy if answers are categorical, or requires normalization.
- We could also use domain-specific criteria – e.g., for a diagnosis question, maybe partially credit if the model named the correct diagnosis among others.

An example of an automated eval: suppose our test set is 100 questions from MedQuAD (which are QA pairs). We can compute the **BLEU score** of the model's answers against the reference answers (treating it like a translation problem from question to answer). However, BLEU might be harsh if the wording differs. **BERTScore** is often better for QA because it uses contextual embeddings to judge if the answer has the same meaning as the reference [36] [35]. BERTScore will tell us, for each answer, how close it is in meaning to the reference (on a 0-1 scale, where closer to 1 is better). We can average that as an overall quality metric. If our model is truly good, its BERTScore might be, say, 0.85+ against references, whereas a weaker model might score 0.5.

For multiple-choice, the **accuracy** is straightforward to compute and very interpretable (e.g., "Model X got 85% of MedMCQA questions correct, compared to GPT-4 which got ~90% [37]"). In fact, MedMCQA has a leaderboard; we could see where we stand.

- **Human Evaluation:** For reasoning quality and safety, human experts (or at least medical students/practitioners) should review a sample of the chatbot's answers. They can rate them on criteria like correctness, completeness, clarity, and presence of proper reasoning. This is more labor-intensive, but for a medical chatbot, it's critical. One way to incorporate this systematically is to use existing human-rated benchmarks. For example, there was an evaluation in research where doctors

evaluated ChatGPT and other models on medical answers. If such a benchmark is available (like a set of questions with human ratings for various models), we could use our model on those questions and compare. If not available, we might conduct a small study.

That said, since the user specifically asked *"how LLMs are usually benchmarked anyway?"*, we should mention known evaluation benchmarks: - **MMLU (Medical subjects)**: This is a subset of a broader benchmark, but MMLU includes categories like Medicine, Biology, etc. GPT-4 and others have scores on it. Our model can be tested on the Medicine category of MMLU (which is multiple-choice questions). If we get say 70% right and GPT-4 is 85%, we know where we stand. - **MedQA (USMLE)**: If we have access to a set of USMLE exam questions (the authors of some papers have them), we could see if our model passes. For instance, one study showed GPT-4 exceeded the passing score on USMLE (over 85% correct) while GPT-3.5 was around 60%. We could evaluate our model similarly. - **PubMedQA and BioASQ**: These are benchmarks for biomedical QA. BioASQ, for example, asks factoid questions and expects specific answers. We can use exact match and list accuracy metrics that BioASQ uses (they often use Mean Reciprocal Rank if multiple possible answers). - **Real-world queries**: Possibly take a slice of a forum (like healthtap or reddit AskDocs) and see how our model answers vs. ground truth answers from doctors. But that can be messy and not standardized.

For our immediate purposes, an automated evaluation on held-out Q&A pairs plus a careful check on reasoning is good.

Example script for automated benchmarking:

```
from transformers import pipeline
import evaluate

# Load fine-tuned model
model = AutoModelForCausalLM.from_pretrained("medgemma-27b-med-finetuned",
device_map="auto")
tokenizer = AutoTokenizer.from_pretrained("medgemma-27b-med-finetuned")
pipe = pipeline("text-generation", model=model, tokenizer=tokenizer,
                max_new_tokens=256, temperature=0.0)  # deterministic

# Suppose test_data is a list of dicts with 'question', 'context', 'answer'
predictions = []
references = []
for item in test_data:
    prompt = ""
    if item.get("context"):
        prompt += f"{item['context']}\n"
    prompt += f"Question: {item['question']}\nAnswer:"
    # Generate model's answer
    output = pipe(prompt)[0]['generated_text']
    # Extract just the answer portion from output (depending on how it's
formatted)
    model_answer = output.split("Answer:")[-1].strip()
    predictions.append(model_answer)
```

```
    references.append(item["answer"].strip())

# Use BERTScore to evaluate semantic similarity
bertscore = evaluate.load("bertscore")
results = bertscore.compute(predictions=predictions, references=references,
lang="en")
# results will have precision, recall, F1 lists; we can take mean F1 as overall
mean_bertscore_f1 = sum(results["f1"]) / len(results["f1"])
print(f"Avg BERTScore-F1: {mean_bertscore_f1:.3f}")
```

If the average BERTScore F1 is, say, 0.8 or above, that indicates the model's answers are quite close in content to the reference answers [35] . We could also compute BLEU:

```
bleu = evaluate.load("bleu")
bleu_result = bleu.compute(predictions=predictions, references=[[ref] for ref in
references])
print("BLEU:", bleu_result["bleu"])
```

But BLEU is often low for long answers even if they're correct (because wording differs).

For multiple-choice sets:

```
correct = 0
total = 0
for q in mcq_test:
    model_ans = get_model_answer_choice(q["question"], q["choices"])  # need to
design how model outputs choice
    if model_ans == q["correct"]:
        correct += 1
    total += 1
print("Accuracy:", correct/total)
```

Designing `get_model_answer_choice` could be done by either prompting the model to just output the letter (A/B/C/D) or by having it output an explanation and then parsing the chosen answer. Simpler: prepend "Options: A. ... B. ... C. ... D. ..." to the prompt and then ask "The correct option is". Then pick the letter from the output.

LLM eval harness: There are toolkits like EleutherAI's LM Evaluation Harness that have implementations for many QA tasks. For example, it has tasks like MedMCQA and PubMedQA built-in [38] . We could use those to get a standardized evaluation. This ensures we use the same test splits and metrics as others. For instance:

```
# (This is conceptual CLI)
lm-eval evaluate --model medgemma-27b-med-finetuned --tasks medmcqa pubmedqa
```

This would output accuracy or other metrics defined for those tasks. If the harness is not easily available, our manual approach as above is fine.

- **Semantic similarity vs factual correctness:** One caution – a high similarity score doesn't guarantee factual correctness if both the model and reference are wrong or both have the same misconception. However, since references are presumably correct, similarity to them is a decent proxy. For an extra layer, we could have a separate verification stage: e.g., feed the question and the model's answer to an external checker (like an information retrieval system or even an LLM evaluator) to see if claims are correct. That's advanced and usually not done in standard benchmarks. Usually, we trust the reference-based metrics or use human eval for factual accuracy.

Reasoning quality: To specifically measure reasoning, one approach is **to evaluate the chain-of-thought** if the model produces one. For instance, if we fine-tuned it to output a rationale, we can check how often the rationale leads to the correct conclusion. We might create a small set of puzzles or diagnostic questions where the process is important, then manually verify the reasoning steps. In research, sometimes metrics like **step accuracy** (how many intermediate steps are correct) are used.

Given the emphasis on *optimal accuracy and reasoning*, we should use benchmarks that stress both: - MedMCQA or MedQA (exam questions) test **knowledge and reasoning** because many questions require reasoning through symptoms to pick an answer. So accuracy on these is a proxy for reasoning ability in a medical sense. - For pure reasoning (logic/math), one could test something like a step-by-step math word problem or a logical reasoning puzzle. But that might be outside medical domain. If the chatbot needs reasoning (like dosage calculations, etc.), we can test a few.

Summaries of expected performance: We can say, for instance, after fine-tuning: - The model achieves X% on MedMCQA (compared to baseline Y%). - On a set of 100 MedDialog questions, doctors rated 90% of its answers as correct and helpful. - The model's answers have a BERTScore of 0.85 against reference answers on a validation set, indicating a high semantic similarity to correct answers (embedding-based metrics like BERTScore are effective for evaluating QA where semantic accuracy matters more than exact wording [35]).

This combination of metrics will give a well-rounded picture.

7. Multi-Model Inference and Round-Robin API Utilization

Finally, in deploying our chatbot system, we have the opportunity to **fuse multiple models via their APIs** to optimize performance and cost. The idea is: - Use the **Gemini API** for heavy-duty queries (where the highest accuracy or creativity is needed, given Gemini is presumably a very strong model). - Use the **NVIDIA API** with smaller LLMs for simpler queries or supportive tasks (to save on usage of the more expensive Gemini calls). - Use a **Round-Robin strategy** for API keys to distribute load evenly and avoid hitting rate limits or exhausting any single key's quota. The user indicated we have multiple keys for each service (`Gemini_COS30018_1` … `_5` and `NVIDIA_COS30018_1` … `_5`).

Round-Robin API key usage: If we have 5 keys for Gemini, we can cycle through them for each request. That way, each key handles 1/5 of the requests, effectively multiplying our throughput and reducing the chance of one key hitting a cap. Similarly for NVIDIA's keys.

Smart routing between models: We want to decide when to use the big model vs the smaller model. Possible criteria: - **Complexity of query:** We can estimate complexity by the length of the question, the presence of certain keywords, or even by running a quick classification (like if the question requires reasoning or specialized knowledge). For example, a question like "What are the side effects of ibuprofen?" is pretty straightforward lookup (a smaller model or even a database could handle). But "My 5-year-old has a fever and rash, what could be the cause?" is more complex (requires differential diagnosis) – we might route that to Gemini. - **Conversation stage:** If this is part of a longer dialog, maybe stick to one model for consistency within a session. Alternatively, use big model for initial complex diagnosis, but use smaller model for follow-up clarifications. - **Latency/cost considerations:** Smaller models likely have faster response and lower cost. So for high-volume simple queries, prefer them.

We can implement a simple heuristic: if the user's query (after tokenization) is short and doesn't contain many medical entities, or if it looks like a factual question with a known answer, use the smaller model. If it's long or has multiple parts or critical tone (e.g., "should I go to ER?"), use the larger model.

Let's illustrate with pseudocode how to set up **round-robin and routing**:

```
import itertools
import os
import requests  # for API calls

# List of API keys from environment variables
gemini_keys = [os.getenv(f"Gemini_COS30018_{i}") for i in range(1, 6)]
nvidia_keys = [os.getenv(f"NVIDIA_COS30018_{i}") for i in range(1, 6)]

# Create round-robin iterators for keys
gemini_key_cycle = itertools.cycle(gemini_keys)
nvidia_key_cycle = itertools.cycle(nvidia_keys)

# Function to call Gemini API (placeholder, actual API details needed)
def call_gemini_api(prompt, model="gemini-2.5-flash"):
    api_key = next(gemini_key_cycle)
    url = "https://api.gemini.service/v1/completions"  # hypothetical endpoint
    headers = {"Authorization": f"Bearer {api_key}"}
    payload = {
        "model": model,
        "prompt": prompt,
        "max_tokens": 512
    }
    response = requests.post(url, json=payload, headers=headers)
    result = response.json()
    return result.get("completion")  # assuming the API returns a JSON with
'completion'

# Function to call NVIDIA API
def call_nvidia_api(prompt, model="fast-chat-model"):
```

```
    api_key = next(nvidia_key_cycle)
    url = "https://api.nvidia.service/v1/completions"
    headers = {"Authorization": f"Bearer {api_key}"}
    payload = {
        "model": model,
        "prompt": prompt,
        "max_tokens": 512
    }
    response = requests.post(url, json=payload, headers=headers)
    result = response.json()
    return result.get("text")

# Router logic to choose which API to use
def get_chatbot_response(user_query):
    # Simple routing criterion: length of query and presence of keywords
    query_length = len(user_query.split())
    # Example condition: if query is short and looks like a factual question
    simple_triggers = ["side effect", "what is", "define", "symptom of", "dose
of"]
    if query_length < 15 or any(kw in user_query.lower() for kw in
simple_triggers):
        # Use the smaller NVIDIA model for a quick answer
        model_name = "slm-2.7b"  # assume an SLM (Small Language Model) of 2.7B
as an example
        response = call_nvidia_api(user_query, model=model_name)
    else:
        # Use Gemini for complex query
        response = call_gemini_api(user_query, model="gemini-2.5-flash")
    return response

# Example usage:
user_question = "What are the symptoms of appendicitis?"
answer = get_chatbot_response(user_question)
print("Assistant:", answer)
```

In this script: - We use `itertools.cycle` to easily rotate through the API keys each time `next()` is called, implementing the round-robin. This ensures we don't overuse one key. - `call_gemini_api` and `call_nvidia_api` are placeholders – in practice, we need the actual endpoint URLs and request format. But the idea is we include the `api_key` in the header for authorization. - The `get_chatbot_response` function routes the query. The logic here is very basic; in reality, we could incorporate more sophisticated checks: - We might parse the question (maybe using a cheap model or regex) to detect if it's a yes/no question, a "what is" definition question, a personal medical advice question, etc. Simple definitional or factual questions could go to a smaller model or even a curated knowledge base, whereas personal complex ones go to big model. - We could maintain a small knowledge base for drug info or common definitions and answer from that without any LLM (that's a form of retrieval augmentation). But since the question focuses on LLM usage, we stick to LLMs. - We also consider using different *models* from the NVIDIA API. Perhaps NVIDIA offers a range like a 1.3B model (very fast, for extremely simple tasks), a 13B model,

etc. They might label them differently (maybe `"fast-chat"` vs `"high-accuracy-chat"`). We'd choose accordingly. The user mentioned "smaller LLM/SLM models to substitute tasks that don't need to be on higher models" – this implies something like using a lightweight model for trivial tasks. Indeed, if a user just asks "What's 2+2?" or "Define hypertension", a small model can do that. Save Gemini for when user asks "I have these 10 symptoms, what should I do?" which requires more. - **Round-robin keys** protect us from hitting limits. For example, if Gemini API allows, say, 60 requests/min per key, with 5 keys we can do 300 requests/min theoretically. It also helps in case one key gets temporarily throttled.

Ensuring consistency: One challenge when using multiple models is consistency in style and persona. The user experience might feel jarring if one response is very terse (from a small model) and next is very verbose (from Gemini). To mitigate that, we can enforce a style guideline via the prompts. For example, always prefix the user query to the model with a system message like: *"You are a compassionate medical assistant who speaks in a clear, concise manner."* If both models get similar system instructions, their outputs will be more aligned in tone. We should craft these prompts carefully and include them in the `prompt` field for each API call if the API supports system messages (OpenAI and likely others have a role-based prompt format).

Failover: If the smaller model yields an uncertain answer (maybe it outputs "I'm not sure"), we could have logic to automatically route that question to the bigger model. This way, if the small model fails, the big model can handle it. We could detect uncertainty by keywords (like "not sure", "I don't know") in the small model's output. This ensures quality – the user won't get a poor answer just because the router guessed wrong initially.

Benchmarking the multi-model system: We might also simulate some queries through the get_chatbot_response function to ensure that the routing logic is performing well (e.g., measuring how often it chooses small vs big model and whether that was the correct choice in hindsight). This is more of a system integration test.

In conclusion, using a **combination of models** in deployment can give us the **best of both worlds**: - Lower-tier models handle simple queries fast and cheaply. - Top-tier model handles complex cases with high accuracy. - Load is balanced across multiple API keys to avoid slow-downs. - The system as a whole is more scalable and cost-efficient.

By following all the steps above – leveraging rich datasets, augmenting the data, distilling knowledge from powerful models, fine-tuning our student with LoRA and reinforcing its correctness with GRPO, and then deploying intelligently with multiple models – we aim to achieve **optimal accuracy and robustness** for the medical chatbot. The model will be grounded in real medical data (ensuring factual correctness) and endowed with strong reasoning ability (both through the diverse training data and the RL fine-tuning). Finally, our evaluation will be based on real benchmarks and metrics, giving us confidence in the chatbot's performance in a practical setting.

Sources:

- MedDialog dataset description (English, 0.26M dialogues from HealthcareMagic/iCliniq) [1] ; Medical dialogue dataset survey [2] .

- MedQuAD NIH QA dataset (47,457 Q&A pairs) [8].
- MedMCQA exam dataset (194k questions across 21 subjects) [9].
- PubMedQA dataset (research questions with expert and generated answers) [10].
- DrugEHRQA medication QA dataset (70k+ Q&A pairs from EHR data) [11] [12].
- MIRIAD large-scale medical Q&A dataset (4.4M–5.8M LLM-generated, literature-grounded pairs) [14] [13].
- Unsloth documentation on mixing reasoning vs non-reasoning data [18].
- Use of Alpaca (52k GPT-generated instructions) for distillation [20].
- Google's discussion on "distilling step-by-step" (using LLM rationales to train smaller models) [21] [22].
- GRPO described as integrating reinforcement learning with fine-tuning and using adaptive rewards [33]; Unsloth's proximity-based reward for correct answers [28].
- HuggingFace PEFT (LoRA) documentation on adding LoRA adapters to models [34].
- Common evaluation metrics: BERTScore for QA/summarization evaluation focusing on semantic accuracy [35].
- Analytics Vidhya on evaluation: embedding-based metrics like BERTScore are useful for QA where exact wording may differ [36] [35].
- LM evaluation harness mention for MedDialog tasks [38] (illustrating available tasks).

---

[1] bigbio/meddialog · Datasets at Hugging Face
https://huggingface.co/datasets/bigbio/meddialog

[2] [5] [6] [7] [10] [15] A survey of datasets in medicine for large language models
https://www.oaepublish.com/articles/ir.2024.27

[3] A dataset of simulated patient-physician medical interviews ... - Nature
https://www.nature.com/articles/s41597-022-01423-1

[4] MedDialog: Large-scale Medical Dialogue Datasets - ACL Anthology
https://aclanthology.org/2020.emnlp-main.743/

[8] GitHub - abachaa/MedQuAD: Medical Question Answering Dataset of 47,457 QA pairs created from 12 NIH websites
https://github.com/abachaa/MedQuAD

[9] MedMCQA Homepage
https://medmcqa.github.io/

[11] [12] DrugEHRQA: A Question Answering Dataset on Structured and Unstructured Electronic Health Records For Medicine Related Queries v1.0.0
https://physionet.org/content/drugehrqa/1.0.0/

[13] [14] MIRIAD: Augmenting LLMs with millions of medical query-response pairs
https://arxiv.org/html/2506.06091v2

[16] [17] A dataset of simulated patient-physician medical interviews with a focus on respiratory cases | Scientific Data
https://www.nature.com/articles/s41597-022-01423-1?error=cookies_not_supported&code=483a564b-3900-4121-ad53-a2d9a77bc58d

[18] [28] [30] Qwen3: How to Run & Fine-tune | Unsloth Documentation
https://docs.unsloth.ai/basics/qwen3-how-to-run-and-fine-tune

19 Data Augmentation for Enhanced Language Model Training
https://www.rohan-paul.com/p/data-augmentation-for-enhanced-language

20 Alpaca: A Strong, Replicable Instruction-Following Model
https://crfm.stanford.edu/2023/03/13/alpaca.html

21 22 23 Distilling step-by-step: Outperforming larger language models with less training
https://research.google/blog/distilling-step-by-step-outperforming-larger-language-models-with-less-training-data-and-smaller-model-sizes/

24 LoRA - Hugging Face
https://huggingface.co/docs/diffusers/training/lora

25 Efficient Large Language Model training with LoRA and Hugging Face
https://www.philschmid.de/fine-tune-flan-t5-peft

26 29 31 32 33 34 Fine-Tuning DeepSeek-7B with GRPO: A Comprehensive Guide | by Rajveer Rathod | Medium
https://medium.com/@rajveer.rathod1301/fine-tuning-deepseek-7b-with-grpo-a-comprehensive-guide-1b7a89ae21b1

27 Mastering LLM Fine-Tuning: GRPO, PPO, and DPO Compared
https://pub.towardsai.net/mastering-llm-fine-tuning-grpo-ppo-and-dpo-compared-e362257d4036

35 What is BERTScore or other embedding-based metrics, and can ...
https://zilliz.com/ai-faq/what-is-bertscore-or-other-embeddingbased-metrics-and-can-they-be-helpful-in-evaluating-the-similarity-between-a-generated-answer-and-a-reference-answer-or-source-text

36 What is BERTScore or other embedding-based metrics, and can ...
https://milvus.io/ai-quick-reference/what-is-bertscore-or-other-embeddingbased-metrics-and-can-they-be-helpful-in-evaluating-the-similarity-between-a-generated-answer-and-a-reference-answer-or-source-text

37 Gemma vs Mistral for Medical Domain Fine-Tune, an Exploration
https://matthewchung74.medium.com/gemma-vs-mistral-for-medical-domain-fine-tune-an-exploration-4e18540bed00

38 lm-evaluation-harness/lm_eval/tasks/meddialog/README.md at main
https://github.com/EleutherAI/lm-evaluation-harness/blob/main/lm_eval/tasks/meddialog/README.md